Riksdagen i siffror

En dataplattform för enklare insyn i demokratin

Bakgrund & problem

Riksdagens öppna data är svår att ta till sig

- API:er sprids över flera endpoints
- Data är ostrukturerad och ej filtrerad
- Ingen samlad vy för medborgare eller journalister
- Svårt att följa hur ledamöter faktiskt agerar

Vår lösning

- Automatiserad datainsamling var 24:e timme
- ETL-pipeline med rensning, flaggning & validering
- Kafka-streaming för realtidsflöde
- Interaktivt dashboard för alla

Orkestrering — main.py

- FastAPI lifespan startar hela pipelinen vid uppstart
- APScheduler triggar fetch var 24:e timme per endpoint
- Kafka-consumer körs parallellt i bakgrundstråd (threading.Thread)

ETL-pipeline

Extract

- HTTP GET mot Riksdagens API
- json_normalize → DataFrame
- format_files städar kolumnnamn
- Sparas som CSV + streamar till Kafka

Transform

- clean.py — normaliserar typer
- flag.py — lägger till flag_reason
- reject.py — tar bort interna kolumner
- Null-ID och felaktiga format avvisas

Load

- Kafka consumer pollar topics
- Upsert till Supabase per rad
- Batch-läge via __main__ för backfill
- 5 tabeller: members, votes, speeches...

Datakvalitet och validering

Valideringsflöde per dataset:

- Rensa (clean.py) — normalisera kolumnnamn, typer, whitespace
- Flagga (flag.py) — identifiera misstänkta rader utan att ta bort dem
- Avvisa (reject.py) — ta bort null-ID, felformat, interna fält, 100% null-kolumner

Teknisk stack

FastAPI

Asynkront, lätt att orkestrerar lifespan + scheduler

Apache Kafka

Frikopplad streaming — producer och consumer kan skalas oberoende

Supabase

Managed Postgres med REST-API, enkel upstart, gratis tier

Pandas

Kraftfull datamanipulation för ETL-stegen

Streamlit

Snabb frontend utan frontend-kod — Python hela vägen

Docker Compose

Reproducerbar miljö, ett kommando startar hela stacken

Testning

Vi testade varje lager i pipelinen för att säkerställa korrekthet och robusthet.

Enhetstester

- pytest täcker clean.py, flag.py och reject.py
- Null och typer testas
- Kafka och Supabase mockas
- Körs vid varje commit

Integrationstester

- ETL-flödet testas från start till slut
- Data når Supabase korrekt
- Upsert-logik kontrolleras mot dubletter
- Testas mot verklig API-respons

Datakvalitetstester

- Inga null-ID:n når databasen
- flag_reason kontrolleras
- Avvisade rader räknas per körning
- Avvikelser loggas

Dokumentation

README

- Installationsinstruktioner steg för steg
- docker compose up startar hela stacken
- Miljövariabler och .env-konfiguration
- Länk till Supabase-schema
- länk till dashboard.

Kodstruktur

- Koden är uppdelad i tre mappar: extract, transform och load
- Varje mapp gör en sak
- main.py startar och styr allt
- Kommentarer finns i koden där det behövs

Datamodell

- Flera tabeller i Supabase
- Relationer och primärnycklar definierade
- Flagg-kolumner för datakvalitet
- Schema exporterat från Supabase

Agilt arbetssätt

Vi har arbetat enligt Scrum med korta iterationer och kontinuerlig leverans:

Sprint Planning

Väljer uppgifter från backlog

→

Sprint (1–2 v)

Bygger, testar och itererar

→

Sprint Review

Demo av levererat

→

Retro-spektiv

Vad funkar?
Vad förbättras?

Kontinuerlig förbättring — varje sprint bygger vidare på föregående

Verktyg: GitHub för versionshantering · Daily standups · Backlog i issues · Sprintlängd: 1 vecka

Dashboard – Vad Kunden ser

Streamlit-dashboarden ger användaren fem vyer att utforska riksdagsdata:

Startsida

KPI-kort med totalt antal ledamöter, möten och beslut — en snabb överblick av riksdagens aktivitet

Voteringar

Filtrera omröstningar per parti och datum. Se hur varje parti röstade i varje fråga

Anföranden

Sök och filtrera tal och debatter från kammaren per ledamot, datum och ämne

Kalender

Kommande riksdagsmöten och ärenden — alltid uppdaterat

Ledamöter

Filtrera alla 349 ledamöter per parti och valkrets

DEMO

REFLEKTIONER FRÅN TEAMET